Reporte Técnico de Actividades Práctico-Experimentales Nro. 004

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	**Alejandro Padilla**
Asignatura	Simulación
Ciclo	5A
Unidad	U1: Introducción a la Simulación
Resultado de aprendizaje de la unidad	R1. Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	004
Tipo	Individual
Título de la Práctica	Análisis comparativo de herramientas de simulación mediante un modelo común
Nombre del Docente	José O. Guamán Q.
Fecha	23 de octubre de 2026

2. Objetivo(s) de la Práctica:

Comparar Python con SimPy y NetLogo como herramientas de simulación mediante un mismo modelo epidemiológico SIR, analizando la sintaxis, la facilidad de implementación y la visualización de cada una.

- Implementar el modelo SIR por individuos en Python con SimPy y en NetLogo, con los mismos parámetros y la misma regla de contagio.
- Integrar ambas herramientas mediante un archivo CSV: Python genera los datos y NetLogo los importa y los grafica junto a su propia simulación.
- Comparar los resultados de una corrida y de 30 réplicas con la solución de las ecuaciones diferenciales, y explicar las diferencias que produce la generación de números aleatorios.
- Completar la tabla comparativa de las dos herramientas con la experiencia obtenida durante la implementación.

3. Materiales y reactivos:

- Computador.
- Python 3 con entorno virtual (Python 3.14).
- NetLogo 7.0.4.
- Archivos CSV de intercambio: resultados_python.csv y resultados_netlogo.csv.
- Java/JDK (indicado en la guía): no se utilizó. El grupo y el alcance acordado con el docente limitaron la práctica a Python/SimPy y NetLogo.

4. Equipos y herramientas

- SimPy 4.1.2.
- NumPy 2.5.4 y Matplotlib 3.11.2.
- pytest 9.1.1 (9 pruebas del modelo).
- NetLogo 7.0.4, con BehaviorSpace en modo sin interfaz para las réplicas y los tiempos.
- Visual Studio Code.
- Sistema operativo Linux.

5. Procedimiento / Metodología

Inicio

- Se revisó el modelo SIR de la guía: una población cerrada de N = S + I + R individuos que pasan de susceptibles a infectados y de infectados a recuperados. La Tabla 1 resume los parámetros comunes, que se usaron sin cambios en las dos herramientas.

Tabla 1. Parámetros comunes de la simulación.

Parámetro	Valor	Significado
N	100	Población total
I(0) / S(0)	5 / 95	Infectados y susceptibles iniciales
β	0.30	Tasa de infección
γ	0.10	Tasa de recuperación
Tiempo	100	Pasos de tiempo simulados (t = 0, 1, ..., 100)
R_0	3	Número reproductivo básico, $R_0 = \beta / \gamma = 0.30 / 0.10$

El modelo continuo de referencia es el sistema de ecuaciones diferenciales de la guía:

$$dS/dt = -\beta \cdot S \cdot I / N$$
$$dI/dt = \beta \cdot S \cdot I / N - \gamma \cdot I$$
$$dR/dt = \gamma \cdot I$$

- La guía pide una simulación discreta por pasos, no resolver las ecuaciones. Como los infectados iniciales son 5 de 100 y $R_0 = 3 > 1$, se espera una epidemia con un pico de infectados y casi toda la población recuperada al final.

Desarrollo

- **Regla de contacto común.** En cada paso, cada infectado (según el estado al inicio del paso) elige un contacto al azar entre los otros 99 individuos; si ese contacto es susceptible y un número aleatorio es menor que β, se infecta. Luego el infectado se recupera si otro número aleatorio es menor que γ. La actualización es síncrona: se lee el estado actual y se escribe en una copia. Esta mezcla homogénea da $R_0 = \beta / \gamma = 3$, igual que las ecuaciones.

Pseudocódigo de un paso, igual en Python y en NetLogo:

```
PARA t = 0 HASTA 100
    guardar t, contar(S), contar(I), contar(R)
    copia ← estado
    PARA cada individuo i con estado[i] = INFECTADO
        c ← individuo al azar entre los otros 99
        SI estado[c] = SUSCEPTIBLE Y aleatorio < β ENTONCES copia[c] ← INFECTADO
        SI aleatorio < γ ENTONCES copia[i] ← RECUPERADO
    estado ← copia
```

- **Python con SimPy.** La población es un arreglo de NumPy con el estado de cada individuo (0, 1, 2). Un proceso de SimPy guarda los conteos y avanza un paso con env.timeout(1). La semilla se pasa por línea de comandos (--seed, por defecto 42) y alimenta numpy.random.default_rng. El código sigue la arquitectura MVC de la Tabla 2.

Tabla 2. Arquitectura MVC del código en Python.

Carpeta	Archivos	Responsabilidad
model/	config, sir_model, simulation, replications, ode, analysis, csv_io	Parámetros, modelo SimPy, corridas, réplicas, referencia de las ecuaciones, métricas y CSV; no imprime ni grafica
view/	figures, console_view	Figuras de Matplotlib y resumen en consola
controller/	simulation_controller	Lee los argumentos y orquesta modelo y vistas
__main__.py	—	Solo llama al controlador; corre con python -m simulacion_ape y por su ruta
tests/	test_simulation	9 pruebas: CSV de 101 filas, población conservada, semilla, S no crece, β = 0, referencia, réplicas

- **NetLogo.** Cada individuo es una tortuga con las variables state y next-state (copia para la actualización síncrona), dibujada como una persona que camina por una plaza (césped, caminos, fuente, casas y estanque); azul es susceptible, rojo infectado y gris recuperado. Cada tick se divide en subpasos con giros suaves y fotogramas de caminata, esquivando edificios; los infectados caminan a la mitad de velocidad. Cada contagio se dibuja como una línea roja del infectante al contagiado durante 2 ticks; la regla de contacto sigue siendo la mezcla homogénea. Todo el código visual se ejecuta dentro de with-local-randomness (generador aparte), así que no altera ningún sorteo: con la semilla 42 el CSV es idéntico con o sin animación. La interfaz tiene los botones y controles de la Tabla 3, deslizadores, monitores del pico y cinco gráficos.

Tabla 3. Botones y controles del modelo de NetLogo.

Botón o control	Acción
SETUP	Crea 100 tortugas, infecta a 5, fija la semilla y reinicia los gráficos
GO	Ejecuta un paso por tick hasta el tick 100 y redibuja los gráficos
CARGAR PYTHON	Abre resultados_python.csv, lee el encabezado y cada fila, cierra el archivo y dibuja los datos
EXPORTAR NETLOGO	Escribe resultados_netlogo.csv con la serie de la simulación de NetLogo
animacion?, velocidad-animacion	Interruptor y deslizador (1 a 20 subpasos por tick, 8 por defecto); apagado, un solo paso por tick y corrida más rápida

- **Contrato del CSV e integración.** Ambos archivos tienen el encabezado tiempo,susceptibles,infectados,recuperados, 101 filas (t = 0 a 100), enteros, coma como separador y UTF-8. Python escribe resultados_python.csv en la carpeta del modelo de NetLogo; NetLogo solo lo importa y lo grafica, sin generar esos datos. NetLogo exporta resultados_netlogo.csv, que Python lee para dibujar la comparación de las dos herramientas.

Cierre

- Se repitió el experimento con 30 semillas (0 a 29) en cada herramienta y se resolvieron las ecuaciones diferenciales con el método de Euler (dt = 1) como referencia determinista.
- Se midieron los tiempos de ejecución y se completó la tabla comparativa de la guía con lo observado al implementar.

6. Resultados

Las Figuras 1 a 3 corresponden a Python/SimPy; la Figura 4, a NetLogo; la Figura 5 compara las dos herramientas con la misma semilla. Las cifras que se comparan están en la Tabla 4.

Simulación en Python/SimPy

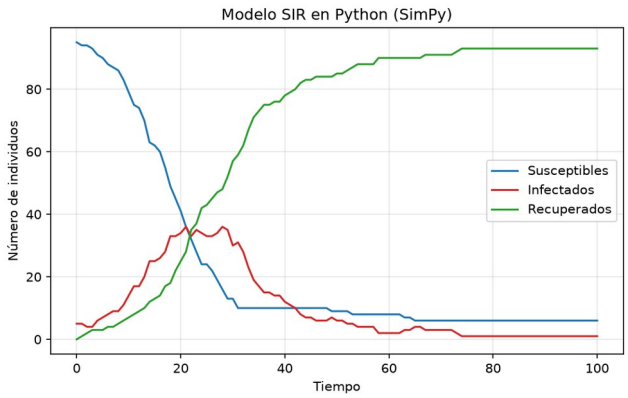


Figura 1. S(t), I(t) y R(t) de Python con semilla 42.

- El pico de infectados es 36 en t = 21.
- Al final quedan 6 susceptibles y 93 recuperados (1 infectado).

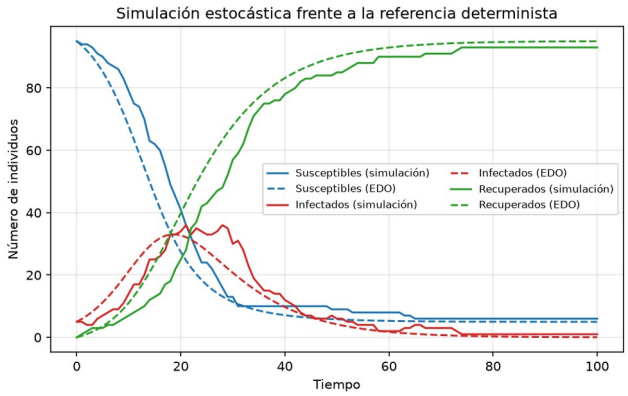


Figura 2. Simulación estocástica de Python frente a las ecuaciones diferenciales (Euler).

- La referencia determinista da un pico de 33.0 en t = 19 y 95.0 recuperados al final.
- La corrida estocástica sigue la misma forma, pero con saltos y un pico más tardío y más alto; la curva continua no los produce.

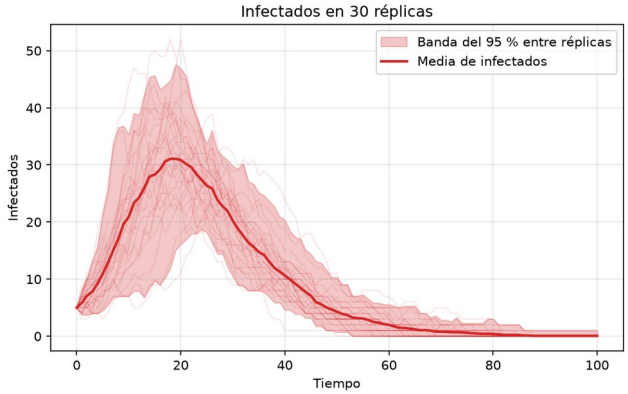


Figura 3. Infectados en 30 réplicas de Python (semillas 0 a 29) y banda del 95 %.

- El pico medio es 36.13 con desviación 6.99; en las réplicas va de 24 a 52 infectados.
- Los recuperados finales promedian 93.27; algunas réplicas se quedan en 70 porque la epidemia se extingue antes de alcanzar a casi todos.

Simulación en NetLogo

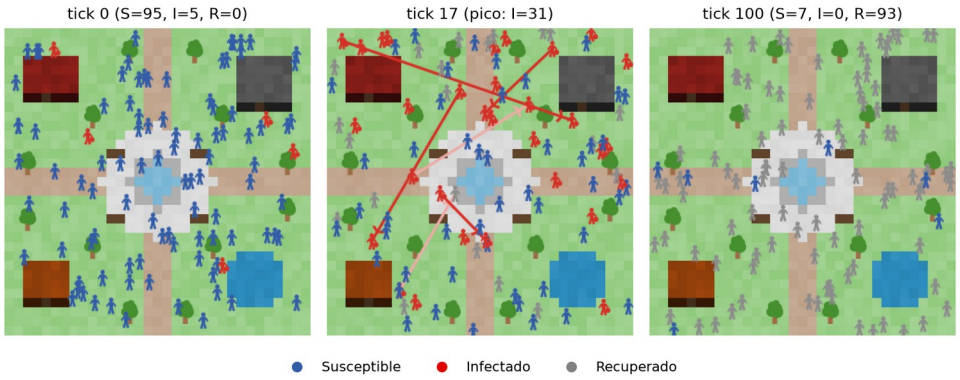


Figura 4. Mundo de NetLogo con semilla 42 en los ticks 0, 17 (pico) y 100: personas en la plaza y líneas rojas de contagio.

- El pico es 31 infectados en el tick 17; al final quedan 7 susceptibles y 93 recuperados.
- La figura es la vista del mundo exportada con export-view; las líneas rojas del tick 17 son contagios recientes (la clara tiene un tick de antigüedad). Los gráficos de la ventana de NetLogo se reproducen en la Figura 5 con los CSV de ambas herramientas.

Comparación entre Python y NetLogo

Tabla 4. Resultados de Python, NetLogo y las ecuaciones diferenciales (media ± desviación estándar en las réplicas).

Métrica	Python semilla 42	NetLogo semilla 42	Python 30 réplicas	NetLogo 30 réplicas	Ecuaciones (Euler)
Pico de infectados	36	31	36.13 ± 6.99	35.33 ± 6.93	33.0
Instante del pico	21	17	19.33 ± 5.06	18.83 ± 5.43	19
Susceptibles finales	6	7	6.67 ± 5.44	8.23 ± 6.46	5.0
Recuperados finales	93	93	93.27 ± 5.46	91.77 ± 6.46	95.0

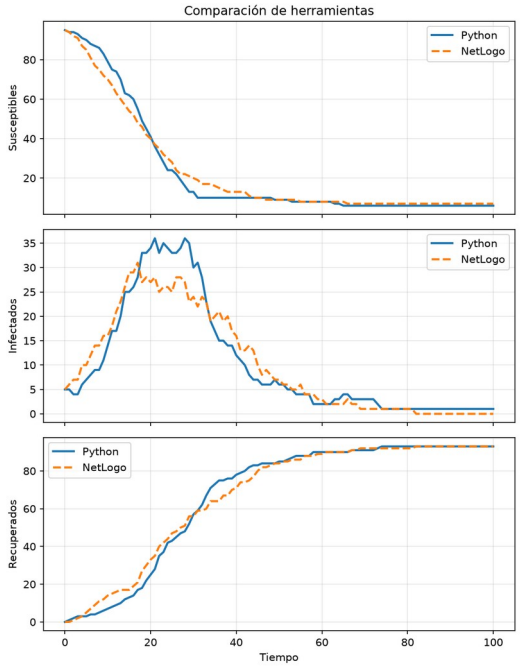


Figura 5. Comparación de susceptibles, infectados y recuperados entre Python y NetLogo (semilla 42).

De la Tabla 4 y la Figura 5 se desprenden tres observaciones:

- **Una corrida difiere, el conjunto no.** Con la misma semilla las curvas se separan (pico 36 frente a 31, cuatro ticks de diferencia), pero con 30 réplicas los promedios son casi iguales: el estadístico t de Welch vale 0.45 para el pico, 0.37 para su instante y 0.97 para los recuperados finales, todos menores que 2 en valor absoluto.

- **Los dos modelos son consistentes con las ecuaciones.** Los picos medios (36.13 y 35.33) están cerca del 33.0 de la referencia; los recuperados finales son algo menores (93.27 y 91.77 frente a 95.0) porque las réplicas que se extinguen antes de contagiar a casi todos bajan el promedio.
- **La variabilidad entre réplicas es grande.** La desviación del pico es cercana a 7 infectados en ambas herramientas; con solo 5 infectados iniciales, el azar de los primeros pasos define la epidemia.

Diferencias por la generación de números aleatorios

- **Generadores distintos.** NumPy usa PCG64 y NetLogo, Mersenne Twister: la misma semilla 42 da secuencias distintas. Los infectados de los primeros ticks son 5, 5, 4, 4, 6, 7, 8 en Python y 5, 6, 7, 7, 10, 10, 12 en NetLogo.
- **Orden de consumo distinto.** Python saca un entero y hasta dos números por infectado; NetLogo recorre a los infectados en orden aleatorio, y cada ask que mezcla a las tortugas también consume números (el movimiento visual usa un generador local y no cuenta). Con el mismo generador, las decisiones también serían otras.
- **Una corrida es una muestra.** El pico de NetLogo (31) y el de Python (36) caen dentro del rango de las réplicas (17 a 48 y 24 a 52): la diferencia es azar, no un error de implementación.
- **Estocástico frente a determinista.** Las ecuaciones dan una sola curva suave; la simulación da una distribución. Por eso conviene comparar herramientas con réplicas y bandas, no con una semilla.

Tabla comparativa de herramientas

La Tabla 5 resume lo observado al implementar el mismo modelo. Los tiempos de la Tabla 6 se midieron en este equipo; NetLogo, sin interfaz y con su reloj interno (timer).

Tabla 5. Comparación de NetLogo y Python/SimPy a partir de la experiencia de implementación.

Característica	NetLogo	Python/SimPy
Facilidad de implementación	339 líneas de código (sin comentarios): unas 185 de contagio, gráficos, importación y exportación y el resto de escenario y animación; botones y controles se arrastran sin programar. Sintaxis propia.	372 líneas en tres capas (modelo, vista, controlador), con réplicas, ecuaciones y figuras. Más código, pero lenguaje conocido, con pruebas y depuración en el editor.
Manejo de agentes	Nativo: tortugas con variables propias, ask y one-of other turtles; el mundo se ve sin escribir código.	Los agentes son un arreglo de estados de NumPy; SimPy aporta el reloj y los procesos, no los agentes.
Generación de datos	Listas y file-type para escribir el CSV (el salto de línea se indica a mano); BehaviorSpace repite corridas y da tablas.	csv y NumPy: un bucle de semillas da las réplicas y las estadísticas, con pocas líneas.
Visualización	Mundo animado y gráficos en vivo sin código extra. Sin interfaz solo exporta la vista y los gráficos, no la ventana.	Matplotlib: figuras estáticas con control total (bandas, comparación con las ecuaciones), a costa de programarlas.

Característica	NetLogo	Python/SimPy
Flexibilidad	Cambiar β, γ o la población es mover un control deslizante; limitado al lenguaje y a las extensiones del programa.	Cualquier análisis (ecuaciones, estadística, pruebas); cambiar un parámetro exige argumentos o configuración.
Tiempo de ejecución	Mediana de 34 ms por corrida sin animación (Tabla 6); la primera tarda más por el arranque de la JVM.	Mediana de 4.5 ms por corrida; la primera, de 18 a 30 ms (Tabla 6).
Integración con otras herramientas	Lee y escribe CSV (extensión csv) y se automatiza con BehaviorSpace; aquí solo importó el CSV de Python.	Escribe y lee el CSV de NetLogo sin problemas; se integra con cualquier biblioteca de Python.

Tabla 6. Tiempos de ejecución medidos (una corrida de 100 pasos).

Medida	Python/SimPy	NetLogo
Primera corrida (semilla 42)	18 a 30 ms	250 a 270 ms
Mediana de 30 corridas en el mismo proceso	4.5 ms	34 ms
Proceso completo con una corrida	0.3 s	2 a 3 s

- **Lectura.** NetLogo tarda unas ocho veces más por corrida, pero su corrida incluye registrar los conteos, mover a las personas y redibujar los gráficos en cada tick (medido con animacion? apagado); el arranque de la JVM domina el proceso completo.
- No se tomaron tiempos de Java: no se implementó, por lo que la comparación es entre dos herramientas.

Repositorio del grupo: https://github.com/AlejandroTatum/simulacion-ape-4-sir

7. Preguntas de Control

- **¿Qué representa el modelo SIR?**

Representa cómo se propaga una infección en una población cerrada que se divide en tres grupos: susceptibles, infectados y recuperados. Cada individuo pasa de S a I y de I a R (S + I → I + I e I → R). En esta práctica es una población de 100 individuos con 5 infectados iniciales.

- **¿Qué representan los estados S, I y R?**

S son los individuos que pueden contagiarse, I los que están infectados y contagian, y R los que ya se recuperaron y no vuelven a contagiarse. Siempre se cumple N = S + I + R; en las corridas S + I + R = 100 en cada paso.

- **¿Qué función cumplen los parámetros β y γ?**

β es la tasa de infección: la probabilidad de que un contacto susceptible se infecte en un paso (0.30). γ es la tasa de recuperación: la probabilidad de que un infectado se recupere en un paso (0.10), así que la infección dura en promedio $1/\gamma = 10$ pasos. Juntos dan $R_0 = \beta / \gamma = 3$.

- **¿Por qué es importante utilizar los mismos parámetros en las herramientas?**

Para que las diferencias se deban a la herramienta y al azar, y no a que se modelaron sistemas distintos. Aquí usé los mismos valores (100, 5, 0.30, 0.10, 100) y la misma regla de contacto en Python y NetLogo; con la misma semilla en ambas, cualquier diferencia queda atribuida al generador y a la implementación.

- **¿Por qué Python y NetLogo pueden producir resultados diferentes con el mismo modelo?**

Porque el modelo es estocástico y cada herramienta tiene su generador de números aleatorios (PCG64 en NumPy, Mersenne Twister en NetLogo) y su propio orden de consumo. Con la semilla 42 el pico fue 36 en Python y 31 en NetLogo, pero con 30 réplicas los promedios fueron 36.13 y 35.33. Lo mismo aplica a Java, que no se implementó.

- **¿Qué ventajas ofrece NetLogo para representar sistemas basados en agentes?**

Los agentes son parte del lenguaje: cada tortuga tiene sus variables, ask les da órdenes y se ve el mundo cambiar de color mientras corre. Además, los controles deslizantes y los gráficos permiten explorar parámetros sin programar. Esto lo comprobé al construir el modelo con 339 líneas.

- **¿Qué ventajas ofrece SimPy para desarrollar simulaciones basadas en eventos?**

Maneja un reloj de simulación y procesos escritos como funciones generadoras que esperan con env.timeout, además de recursos para colas (como en la práctica 3). El tiempo simulado es independiente del tiempo real. En este modelo solo usé un proceso con paso fijo, así que aproveché poco la lógica de eventos.

- **¿Qué ventajas ofrece Java para implementar modelos de simulación?**

Conceptualmente, es un lenguaje compilado y de tipado estático, por lo general más rápido que Python en poblaciones grandes o muchas réplicas, con buen manejo de hilos y un ecosistema maduro. En esta práctica no se implementó (decisión del grupo según el alcance), por lo que no tengo medidas propias.

- **¿Qué función cumple el archivo CSV en la integración Python → NetLogo?**

Es el contrato entre las dos herramientas: un archivo de texto con encabezado fijo y 101 filas de enteros que Python escribe y NetLogo lee. Así NetLogo no necesita ejecutar Python ni conocer su código; solo importa y grafica.

- **¿Qué ventajas presenta separar la generación de datos de la visualización?**

Los datos se generan una vez y se pueden graficar de varias formas sin recalcular; el modelo se prueba sin gráficos (las 9 pruebas no dibujan nada); y cada herramienta hace lo que mejor sabe: Python calcula y NetLogo muestra. El costo es respetar el formato exacto del archivo.

8. Conclusiones

- Se implementó el modelo SIR con los mismos parámetros y la misma regla de contacto en Python/SimPy y en NetLogo. Ambas producen epidemias con un pico y cerca de 93 recuperados al final, y concuerdan con las ecuaciones diferenciales.
- La integración por CSV funcionó: Python generó los datos, NetLogo los importó y graficó, y NetLogo exportó los suyos para la comparación. Separar datos y visualización permitió probar el modelo sin gráficos.
- Con la misma semilla las dos herramientas dan resultados distintos (pico 36 frente a 31) porque usan generadores y órdenes de consumo distintos; con 30 réplicas los promedios coinciden (t de Welch menor que 1.1 en las cuatro métricas).
- NetLogo es más rápido de construir y de explorar de forma visual; Python/SimPy exige más código, pero es más flexible para analizar y automatizar. Python fue unas ocho veces más rápido por corrida en este equipo.
- La práctica se limitó a dos herramientas; la comparación con Java de la guía queda fuera del alcance acordado.

9. Declaración de IA

En esta práctica utilicé un agente de programación (el Gentleman, Pi) basado en el modelo Claude de Anthropic como herramienta de apoyo para leer la guía y proponer la regla de contacto común, escribir el modelo en SimPy con arquitectura MVC y sus 9 pruebas, construir el modelo de NetLogo (botones, gráficos e importación del CSV, personas animadas), generar las figuras, medir las réplicas y los tiempos, y redactar borradores de este informe. Las decisiones de alcance (sin Java), de la regla de contacto, de la semilla 42 y del contenido las tomé yo. Corregí tres errores de la IA: la semilla de NetLogo estaba en 12345 y la pasé a 42 para igualar a Python; el modelo de NetLogo se abrió como modelo vacío en la interfaz, se corrigió y lo comprobé abriéndolo yo mismo; y había generado una parte en Java que descarté por el alcance. Verifiqué cada cifra ejecutando el código y revisé el texto antes de entregarlo.

La Tabla 7 recoge, en orden y de forma textual, los principales prompts que escribí al agente.

Tabla 7. Prompts utilizados con el agente de IA, en orden cronológico

N.°	Prompt utilizado
1	crea el repo del ape 4 grupal como se hizo en el ape 3 y ahi explicas como mis compañeros levantan el codigo, readme e indicaciones todo en español y sencillo
2	en java no hay que hacer
3	solo en phython si hicimos en pyhton
4	el informe aun no, tenemos que pulir tema codigo. mergealo y pushea, luego indicame como funciona y como analizarlos desde vscode
5	lo de netlogo lo ponemos en el repo personal no en el grupal no mergees ahi en el grupal, mientras en paralelo explicame el codigo

10. Bibliografía

Guamán, J. O. (2026). *Diapositivas de la semana 4* [Diapositivas de clase, semana 4]. Asignatura Simulación, Carrera de Computación, Universidad Nacional de Loja.